PARALLEL AND DISTRIBUTED COMPUTING

Semester Project Proposal

Project Title	Large-Scale 2D and 3D Image Convolution Using OpenMP
Parallel Model	OpenMP (shared memory, single multi-core node)

No.	Group Member	Roll No.
1	Syed Ashhad Hassan	23K-0584
2	Saad Shakeel	23K-0614
3	Muhammad Bilal	23K-0026
4	Shaheer Khan	23K-0793

1. Introduction and Problem Description

Convolution is one of the most common operations in image processing, computer vision and medical imaging. To get each output value, we take the values around a pixel (or voxel) and combine them using a small grid of weights called a *kernel*. Different kernels give different results, such as blurring, sharpening, edge detection and noise removal.

In this project we apply **2D convolution** to very large images and **3D convolution** to very large volumes, such as medical scans or stacked video frames. The work grows quickly with the data and the kernel: a 20,000 x 20,000 image has 400 million pixels, and a 31x31 kernel needs 961 multiplications for every one of them. On a single core this takes minutes, which makes it a good problem for parallel computing.

Why it can be parallelized: every output value depends only on the input data, never on other outputs. So the image can be split into rows, columns or tiles and handed to different threads with no communication between them. We use **OpenMP** because all threads share the same memory and can read the whole input image directly.

2. Objectives

- Implement 2D and 3D convolution with many filters and a wide range of kernel sizes.
- Build a sequential version and several OpenMP versions (row-wise, column-wise and tile-based).
- Measure how much faster each parallel version is than the sequential one.
- Find out which splitting method (rows, columns or tiles) works best and explain why.
- Study how thread count, data size, kernel size and scheduling affect speedup and efficiency.

3. Problem Size and Dataset

- **2D test:** a generated grayscale image of 20,000 x 20,000 pixels (400 million pixels, about 400 MB). We will try 30,000 x 30,000 if memory allows.
- **3D test:** a generated volume of 512 x 512 x 512 voxels (about 134 million voxels), and larger if memory allows.
- **Scaling tests:** image sizes of 5,000, 10,000 and 20,000 pixels per side. Kernel sizes from 3x3 up to 31x31 in 2D, and from 3x3x3 up to 11x11x11 in 3D.
- **Batch mode:** thousands of real high-resolution images (for example DIV2K), shared between threads, to check that the filters also work correctly on real photos.
- **Why it is large:** on one core, a large-kernel filter on this much data takes from tens of seconds to several minutes, so the parallel speedup is easy to measure.

4. Filters

The following filters will be implemented. Each one uses a different kernel, and some (such as the median filter) need more work per pixel than others, which helps us see how the type of work affects speedup.

Type	2D Filters	3D Filters
Smoothing	Gaussian blur, box blur	3D Gaussian, 3D mean
Sharpening	Sharpen, emboss	-
Edge detection	Sobel, Prewitt, Laplacian	3D Sobel
Noise removal	Median filter	-

5. Implementation Plan

- **Sequential version:** written in C/C++ for all filters in 2D and 3D. It is the reference for both correctness and speedup.
- **Row-wise parallel version:** each thread gets a group of rows (or slices in 3D) using `#pragma omp parallel for`. We test static, dynamic and guided scheduling.
- **Column-wise parallel version:** each thread gets a group of columns instead. Images are stored row by row in memory, so reading columns jumps across memory. We expect this to be slower because of more cache misses, and we will measure by how much.
- **Tile-based parallel version:** the data is cut into small tiles and each thread processes whole tiles using `collapse(2)`. Tiles keep data close together, so this should use the cache better. We test different tile sizes.
- **Optimizations:** cache blocking (working on small pieces that fit in cache) and SIMD vectorization of the inner loop.
- **Correctness check:** every parallel output is compared value by value with the sequential output to make sure they are identical.

6. Performance Evaluation Plan

Each experiment will be run several times and averaged, with time measured by `omp_get_wtime`. We will vary the number of threads (1, 2, 4, 8 and up to the number of available cores), the data size, the kernel size, the filter type and the tile size.

- **Sequential vs parallel:** the sequential version is compared with every parallel version.
- **Row-wise vs column-wise vs tile-based:** the three splitting methods are compared on the same data, filters and thread counts.
- **Speedup** = sequential time / parallel time. **Efficiency** = speedup / number of threads.
- Execution time, speedup and efficiency are shown as graphs for both 2D and 3D.
- We will discuss load balance, cache and memory bandwidth limits, and why speedup stops growing at higher thread counts.

7. Tools and Environment

C/C++ with OpenMP (GCC), run on a multi-core machine. Timing scripts will automate the experiments, and the graphs will be plotted with Python (Matplotlib).

8. Deliverables

- **Source code:** sequential and OpenMP versions (row-wise, column-wise, tile-based, 2D and 3D), with a Makefile and a README on how to build and run them.
- **Benchmark scripts** that run all experiments and save the timings, and the data generator.
- **Performance graphs:** time, speedup and efficiency against threads, data size, kernel size and tile size, including the row-wise vs column-wise comparison.
- **Correctness report:** comparison of every parallel version with the sequential result, with sample before and after images for each filter.
- **Project report:** design, implementation, results, analysis and conclusions.
- **Viva demonstration:** a live run with different thread counts. Every member will be able to explain the full project.